

DELTA – Střední škola informatiky a ekonomie, s.r.o.

Ke Kamenci 151, 530 03 Pardubice

Bible Memorizer

(Gamifikovaná platforma pro retenci textů)

Autor: Matěj Komárek

Obor: Informační technologie

Třída: 4. B

Školní rok: 2025/2026

Prohlašuji, že jsem maturitní projekt vypracoval(a) samostatně, výhradně s použitím uvedené literatury.

V Pardubicích 30. 3. 2023

(vlastnoruční podpis)

Poděkování

Upřímně děkuji Ing. Monika Borkovcová, Ph.D. za odborné vedení a cenné rady při zpracování tohoto projektu.

Resumé

Práce dokumentuje návrh a vývoj mobilní aplikace Bible Memorizer zaměřené na memorizaci biblických textů. Realizace projektu zahrnuje mobilní aplikaci s podporou offline režimu a backendový server pro synchronizaci a sociální funkce. Mobilní aplikace dále podporuje gamifikaci a učení pomocí intervalového opakování.

Klíčová slova

Programování, gamifikace, intervalové opakování, učení, flutter, nestjs, Bible, memorizace

Keywords

Programming, gamification, spaced repetition, learning, flutter, nestjs, Bible, memorization

Obsah

1	Úvod	8
1.1	Cílová skupina	8
1.2	Terminologie	8
1.2.1	Verš	8
1.2.2	Intervalové opakování	9
2	Technologie	9
2.1	Flutter	9
2.2	NestJS	9
2.3	PostgreSQL	9
2.4	SQLite	10
2.5	Cloudflare Tunnel	10
3	Architektura	10
3.1	Mobilní aplikace	10
3.1.1	Ukládání dat	10
3.1.2	Algoritmy	11
3.2	Server	11
3.2.1	Cloudflare Tunnel	11
3.2.2	Synchronizace	11
4	API a komunikace klient-server	12
4.1	Synchronizační endpointy (SyncController)	12
4.2	Uživatelské endpointy (UserController)	13
4.3	Systémové endpointy (AppController)	13
	Literatura	14

1 Úvod

Ve světě žije přes 2.3 miliardy křesťanů. Což znamená, že více než čtvrtina lidí jsou křesťané. Častým cílem křesťanů je studium Bible a zapamatování jejích částí. Dlouhodobé zapamatování textu ale není jednoduchý úkol. Existují aplikace, které memorizování částí Bible zjednodušují, ale ty jsou často zastaralé, těžké na používání a neaplikují metody učení, které uživateli pomáhají, urychlují a zpříjemňují celý proces.

Za tímto účelem vznikla aplikace Bible Memorizer. Tato aplikace se snaží jejím uživatelům zjednodušit cíl zapamatování specifických biblických textů. K tomu používá dva hlavní principy. První z nich je gamifikace. Aplikace používá rozmanité minihry, které uživatele přirozeně nutí zastavit se nad daným textem a přemýšlet nad ním novým způsobem.

Druhý princip je intervalové opakování. Intervalové opakování je způsob memorování informace na základě postupně rostoucích intervalů. Díky tomu se uživatelem naučený text uživateli zobrazuje pouze občasně pro kontrolu jeho retence. Uživatel pak nemusí trávit tolik času opakováním veršů, které jsou pro něj jednoduché.

1.1 Cílová skupina

Jak lze z úvodu a celé koncepce projektu jednoduše odvodit, cílovou skupinou této aplikace jsou výhradně křesťané, kteří si chtějí memorizovat biblické verše.

Aplikace však není vázána na specifickou denominaci. (církev)

Aplikace zároveň podporuje lokalizaci, díky čemuž umožňuje používání křesťanům z jiných zemí, než jen Česká republika. (Aplikace momentálně podporuje český a anglický jazyk)

1.2 Terminologie

1.2.1 Verš

Verš je část textu Bible. Jeho délka je většinou jedna až dvě věty. Jednotlivé verše tvoří kapitoly a jednotlivé kapitoly tvoří biblické knihy, které pak tvoří Bibli.

Každý verš má svojí unikátní, neměnnou referenci většinou udávanou ve formě "název-knihy číslo-kapitoly:číslo-verše". (např. "Jan 3:16")

1.2.2 Intervalové opakování

Intervalové opakování je způsob učení, který se zakládá na postupně rostoucích intervalech. Vysvětlíme si to na příkladu. Student se chce naučit nazpaměť jeden verš. První den se ho naučí. Druhý den si ho zopakuje. Třetí den vynechá, ale čtvrtý den se ujistí, že si ho stále pamatuje. Tento interval postupně roste, takže student si danou informaci neopakuje každý den. Díky tomu může získaný čas využít na jiné učivo a pojmout tak více informací.

2 Technologie

2.1 Flutter

Pro vývoj klientské části mobilní aplikace byl zvolen framework Flutter. Jeho klíčovým specifikem pro tento projekt je využití vlastního renderovacího enginu, který generuje nativní uživatelské rozhraní a zajišťuje jeho vizuální i funkční konzistenci napříč různými mobilními zařízeními. Tento přístup eliminuje závislost na nativních UI komponentách jednotlivých operačních systémů, čímž se předchází fragmentaci vzhledu a přispívá k lepší přístupnosti aplikace.

2.2 NestJS

Serverová logika (backend) je implementována pomocí frameworku NestJS. Nástroj vynucuje modulární architekturu, která umožňuje systematické rozdělení aplikace do izolovaných domén. Díky tomuto striktnímu strukturování jsou procesy jako autentizace, autorizace a především synchronizace dat bezpečně odděleny a spravovány, což radikálně snižuje riziko vzniku datových nekonzistencí na straně serveru.

2.3 PostgreSQL

Jako primární databázový systém pro běh na serveru je nasazen PostgreSQL. Slouží k trvalému ukládání relačně strukturovaných entit, mezi které patří uživatelská data, vazby přátelství, historie procvičování a žebříčky. Podpora komplexních relací a striktní dodržování pravidel transakčního zpracování zajišťují absolutní datovou integritu a konzistenci při souběžných aktualizacích a synchronizačních požadavcích od vícera klientů.

2.4 SQLite

Pro správu lokálních dat přímo v paměti mobilního zařízení je využita databáze SQLite. Její implementace je nezbytnou podmínkou pro fungování aplikace v offline režimu. Entitní data o uložených verších, harmonogramu opakování a pokroku v učení jsou ukládána do tohoto lokálního úložiště a následně, po detekci obnoveného síťového připojení, asynchronně synchronizována s hlavní databází na serveru. Uživatel tak neztrácí kontinuitu učení ani při výpadcích sítě.

2.5 Cloudflare Tunnel

Infrastrukturní spojení mezi lokálním serverem a sítí internet zajišťuje démon Cloudflare Tunnel. Tato technologie vytváří šifrované odchozí spojení z aplikačního kontejneru přímo do sítě Cloudflare. Serverová aplikace (NestJS) díky tomu může bezpečně přijímat vnější požadavky bez nutnosti otevírání veřejných portů na serverovém firewallu, konfigurace NAT nebo nutnosti přidělení statické veřejné IP adresy, čímž se minimalizuje povrch pro potenciální síťové útoky.

3 Architektura

Architektura systému je navržena jako klient-server, kde klientskou část tvoří mobilní aplikace a logiku zpracování dat centralizuje serverová část. Tyto dvě komponenty spolu komunikují převážně asynchronně, aby byla zachována vysoká dostupnost a možnost offline fungování.

3.1 Mobilní aplikace

Klientská část je zodpovědná za prezentaci uživatelského rozhraní, lokální vyhodnocování algoritmů a dočasnou perzistenci dat.

3.1.1 Ukládání dat

Lokální ukládání dat v mobilní aplikaci je primárně zajištěno pomocí relační databáze SQLite. Toto řešení je klíčové pro zajištění plnohodnotného běhu aplikace v offline režimu, kdy si uživatel může prohlížet a procvičovat uložené verše bez závislosti na momentálním internetovém připojení. Pro účely následné synchronizace je v lokálním databázovém

schématu zavedena vlastnost (*needsSync*), která označuje ty databázové záznamy, jež byly na zařízení nově vytvořeny nebo modifikovány a čekají na odeslání na backend.

3.1.2 Algoritmy

Jádrem logiky aplikace pro plánování učení je implementace principu intervalového opakování (Spaced Repetition). Konkrétně je využit algoritmus SM-2 (SuperMemo-2), který na základě předchozího hodnocení obtížnosti daného verše a zaznamenané úspěšnosti při aktuálním procvičování dynamicky vypočítává datum dalšího opakování. Tímto přístupem dochází k optimalizaci procesu učení – problematické verše jsou uživateli k procvičení předkládány v kratších intervalech, zatímco u dobře osvojených textů se interval do dalšího zobrazení exponenciálně prodlužuje.

3.2 Server

Serverová část aplikačního řešení poskytuje REST API a spravuje primární databázové úložiště. Backend je postaven na frameworku NestJS. Z architektonického hlediska je aplikační kód rozdělen do logických modulů, což umožňuje systematické zapouzdření jednotlivých doménových funkcionalit. Toto modulární členění zajišťuje izolaci business logiky a bezpečné zpracování požadavků.

3.2.1 Cloudflare Tunnel

Pro bezpečné vystavení lokálního serveru do prostředí sítě internet je využit nástroj Cloudflare Tunnel. Tato technologie vytváří zabezpečené odchozí spojení přímo do sítě Cloudflare, čímž eliminuje nutnost otevírání veřejných portů na serverovém firewallu či přiřazování statických veřejných IP adres. Zásadním přínosem tohoto řešení je nativní zajištění šifrované komunikace, kdy je veškerý datový provoz mezi klientskou aplikací a serverem automaticky směrován přes protokol HTTPS, což zaručuje důvěrnost a integritu přenášených dat.

3.2.2 Synchronizace

Synchronizační mechanismus zajišťuje konzistenci dat mezi lokální databází na mobilním zařízení a hlavní serverovou databází. Proces synchronizace je iniciován ze strany klienta. Aplikace vyfiltruje záznamy s vlastností *needsSync* a odešle je formou dávky na server. Pro

řešení případných datových konfliktů – například pokud uživatel procvičoval verše offline na dvou různých zařízeních – je implementován mechanismus „zdroje pravdy“ (source of truth) založený na časovém razítku poslední úpravy záznamu. Server při slučování dat porovná časová razítka (*timestamp* nebo datum procvičení) a do hlavní databáze zapíše tu variantu záznamu, která nese nejnovější časový údaj.

4 API a komunikace klient-server

Architektura programového rozhraní (API) je navržena s ohledem na minimalizaci síťové zátěže a podporu offline běhu mobilní aplikace. Z tohoto důvodu je počet dostupných REST API endpointů záměrně redukován na nezbytné minimum. Přesun dat a udržování stavu mezi klientskou aplikací a serverem je řešen převážně prostřednictvím centralizované synchronizační logiky, namísto tradičního CRUD (Create, Read, Update, Delete) přístupu pro každou datovou entitu zvlášť.

Komunikace je rozdělena do tří hlavních kontrolerů v rámci backendového frameworku NestJS:

4.1 Synchronizační endpointy (SyncController)

Základním pilířem datové výměny jsou dva endpointy obsluhující synchronizační proces. Tyto endpointy pracují na principu dávkového zpracování dat.

- **GET /sync/pull:** Slouží k získání dat ze serveru směrem do klientské aplikace. Klient v parametru dotazu odesílá časové razítko poslední úspěšné synchronizace (*lastSync*). Server na základě tohoto údaje vyhledá a vrátí pouze ty záznamy (uložené verše, historii procvičení, stav přátelství a uživatelská data), které byly vytvořeny, upraveny nebo smazány po zadaném čase (s využitím tzv. soft-delete příznaku *deletedAt*).
- **POST /sync/push:** Zajišťuje nahrání lokálních změn z klientského zařízení na server. Klient odesílá objekt (DTO), který obsahuje pole nových či upravených entit. Server následně přijatá data iteruje a bezpečně ukládá do databáze, přičemž ověřuje identitu odesílatele pomocí přiřazeného *userId*.

4.2 Uživatelské endpointy (UserController)

Tento kontroler spravuje operace vázané na uživatelský profil, které nelze efektivně řešit pouhou synchronizací stavu.

- **GET /user/lookup:** Endpoint pro vyhledávání uživatelů za účelem navázání přátelství. K vyhledání slouží unikátní šestimístný alfanumerický identifikátor (*friend-Code*), čímž je chráněno soukromí uživatelů a skryta jejich e-mailová adresa.
- **GET /user:** Vrací aktuální profil autentizovaného uživatele.
- **DELETE /user/me:** Zajišťuje kompletní a bezpečné odstranění uživatelského účtu. Operace je prováděna v rámci jedné databázové transakce, což garantuje korektní smazání veškerých navázaných dat (veršů, procvičení i přátelství) bez vzniku nekonzistencí či sirotčích záznamů.

4.3 Systémové endpointy (AppController)

- **GET /health:** Standardní endpoint pro monitorování dostupnosti a životního cyklu aplikačního kontejneru.

Reference

- [1] Citace

Seznam obrázků

Seznam příloh

- Příloha 1

Přílohy